TP7: MPI Communicators

Parallel and Distributed Programming

Ikram Benfellah

March 16, 2026

1 Introduction

This lab explores MPI Cartesian communicators through two exercises: implementing Conway's Game of Life on a 2D periodic grid, and solving the 2D Poisson equation using Jacobi iteration with 2D domain decomposition. Both exercises use `MPI_Cart_create`, `MPI_Cart_shift`, and ghost cell exchanges via `MPI_Sendrecv`.

2 Exercise 1: Conway's Game of Life

2.1 Objective

Simulate Conway's Game of Life on a distributed 2D grid using a Cartesian MPI topology with periodic boundary conditions.

2.2 Implementation

2.2.1 Compilation and Execution

```
module load OpenMPI/5.0.7-GCC
mpicc -o game_of_life game_of_life.c
mpirun -np 4 ./game_of_life 20 20 10
```

2.2.2 Algorithm

1. `MPI_Dims_create(nprocs, 2, dims)` computes a suitable $P_x \times P_y$ process grid.
2. `MPI_Cart_create` creates a 2D Cartesian communicator with periodic boundaries.
3. Each process gets its coordinates via `MPI_Cart_coords` and allocates a local subgrid of size local_nx × local_ny plus a 1-cell halo border.
4. `MPI_Cart_shift` along dimensions 0 and 1 identifies the four direct neighbors (N, S, E, W).
5. At each generation:
   - North/South ghost **rows** are exchanged via `MPI_Sendrecv` (contiguous data).

- East/West ghost **columns** are exchanged using an `MPI_Type_vector` derived type. Columns are exchanged *after* rows so that corner ghost cells are filled correctly (needed for the 8-neighbor stencil).
- Conway's rules are applied on interior cells.

6. The final grid is gathered on rank 0 for display.

2.3 Topology

With 4 processes, `MPI_Dims_create` produces a 2×2 grid. Since boundaries are periodic, every process has 4 valid neighbors (wrap-around):

```
Rank 0 coords=(0,0): N=2 S=2 W=1 E=1
Rank 1 coords=(0,1): N=3 S=3 W=0 E=0
Rank 2 coords=(1,0): N=0 S=0 W=3 E=3
Rank 3 coords=(1,1): N=1 S=1 W=2 E=2
```

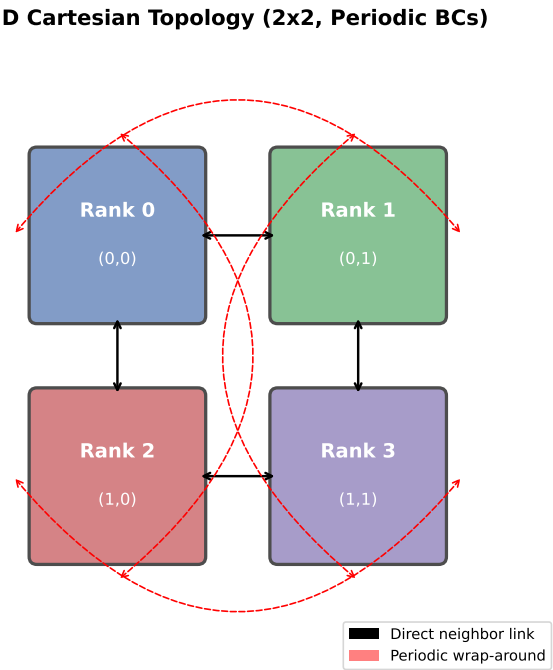

Figure 1: 2D Cartesian topology with periodic boundary conditions. Black arrows show direct neighbor links; red dashed arrows show periodic wrap-around connections.

2.4 Results

The simulation ran on a 20×20 grid for 10 generations with 4 MPI processes (each handling a 10×10 subgrid).

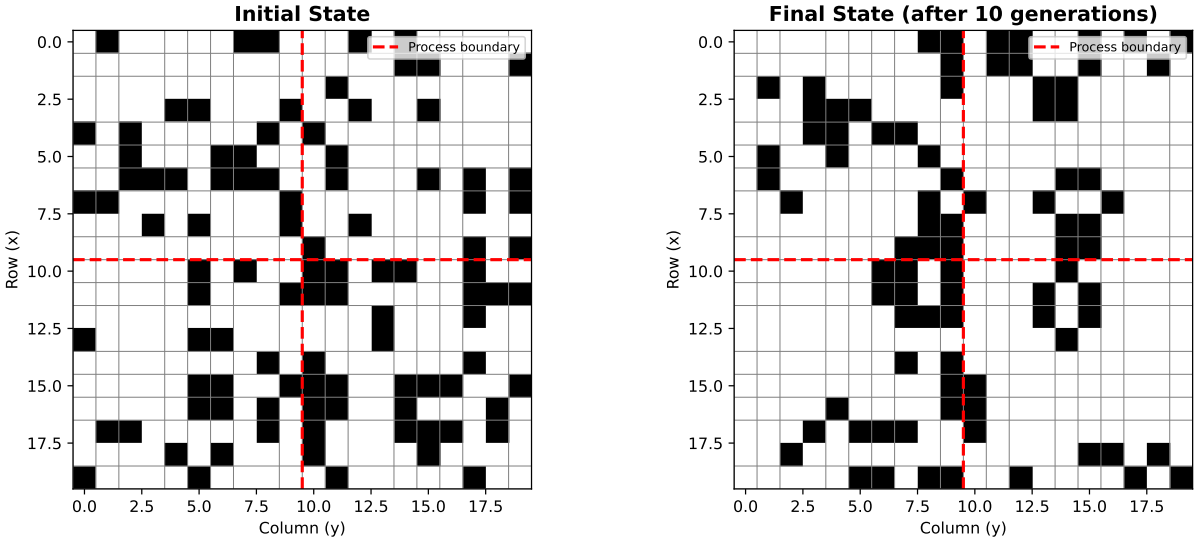

Figure 2: Initial state (∼20% random density) and final state after 10 generations. Red dashed lines indicate process boundaries.

Metric	Value
Global grid size	20×20 (400 cells)
Process grid	2×2 (4 processes)
Live cells (initial)	∼80 / 400
Live cells (final)	85 / 400
Execution time	0.0030 s

Table 1: Game of Life execution summary.

2.5 Observations

- Periodic boundaries ensure no edge effects: cells on the border wrap around and interact with cells on the opposite side, producing a toroidal topology.
- Columns are non-contiguous in row-major memory layout, therefore `MPI_Type_vector` is required for the East/West ghost exchange, unlike rows which are contiguous.
- Corner ghost cells are correctly filled by performing row exchanges first, then column exchanges on the full height (including ghost rows).
- The output is consistent across runs with the same seed, confirming deterministic behavior.

3 Exercise 2: 2D Poisson Equation

3.1 Objective

Solve the 2D Poisson equation $-\Delta u = f$ on the unit square $[0, 1]^2$ using the Jacobi iterative method with a 2D MPI Cartesian domain decomposition.

The exact solution is $u(x, y) = xy(x-1)(y-1)$ with right-hand side $f(x, y) = 2(x^2 - x + y^2 - y)$.

3.2 Implementation

3.2.1 Compilation and Execution

```
module load OpenMPI/5.0.7-GCC
mpicc -o poisson poisson.c -lm
mpirun -np 4 ./poisson 12 10
```

3.2.2 Algorithm

1. `MPI_Dims_create` + `MPI_Cart_create` build a 2D Cartesian communicator (**non-periodic**, since Dirichlet boundary conditions apply).

2. The domain is decomposed in **both** x and y directions. Each process computes local bounds (sx, ex, sy, ey) from its Cartesian coordinates.

3. `MPI_Cart_shift` identifies N/S/E/W neighbors. Boundary processes have `MPI_PROC_NULL` neighbors (value -2 in OpenMPI).

4. The Jacobi update uses three precomputed coefficients from the grid spacing $h_x = \frac{1}{n_{tx}+1}$, $h_y = \frac{1}{n_{ty}+1}$:

$$c_0 = \frac{h_x^2 h_y^2}{2(h_x^2 + h_y^2)}, \quad c_1 = \frac{1}{h_x^2}, \quad c_2 = \frac{1}{h_y^2}$$

$$u_{i,j}^{(n+1)} = c_0 \Big[c_1 \big(u_{i+1,j}^{(n)} + u_{i-1,j}^{(n)} \big) + c_2 \big(u_{i,j+1}^{(n)} + u_{i,j-1}^{(n)} \big) - f_{i,j} \Big]$$

5. Convergence is checked via `MPI_Allreduce`: $\|u^{(n+1)} - u^{(n)}\|_2 < 10^{-6}$.

3.2.3 Pseudo-code

```
MPI_Dims_create -> 2D process grid (Px, Py)
MPI_Cart_create -> Cartesian communicator (non-periodic)
MPI_Cart_shift -> identify N, S, E, W neighbors

initialize u, u_new, u_exact, f

for iter = 1 to max_iter:
    exchange ghost rows (N/S, contiguous, MPI_Sendrecv)
    exchange ghost columns (E/W, MPI_Type_vector, MPI_Sendrecv)
    compute Jacobi update (u -> u_new)
    local_err = sum( (u_new - u)^2 )
    MPI_Allreduce(local_err -> global_err, MPI_SUM)
    swap(u, u_new)
    if sqrt(global_err) < tolerance: break
```

3.3 Domain Decomposition

With ntx = 12, nty = 10, and 4 processes in a 2×2 grid:

```
Rank 0: x from 1 to 6, y from 1 to 5 neighbors: N=-2 E=1 S=2 W=-2
Rank 1: x from 1 to 6, y from 6 to 10 neighbors: N=-2 E=-2 S=3 W=0
Rank 2: x from 7 to 12, y from 1 to 5 neighbors: N=0 E=3 S=-2 W=-2
Rank 3: x from 7 to 12, y from 6 to 10 neighbors: N=1 E=-2 S=-2 W=2
```

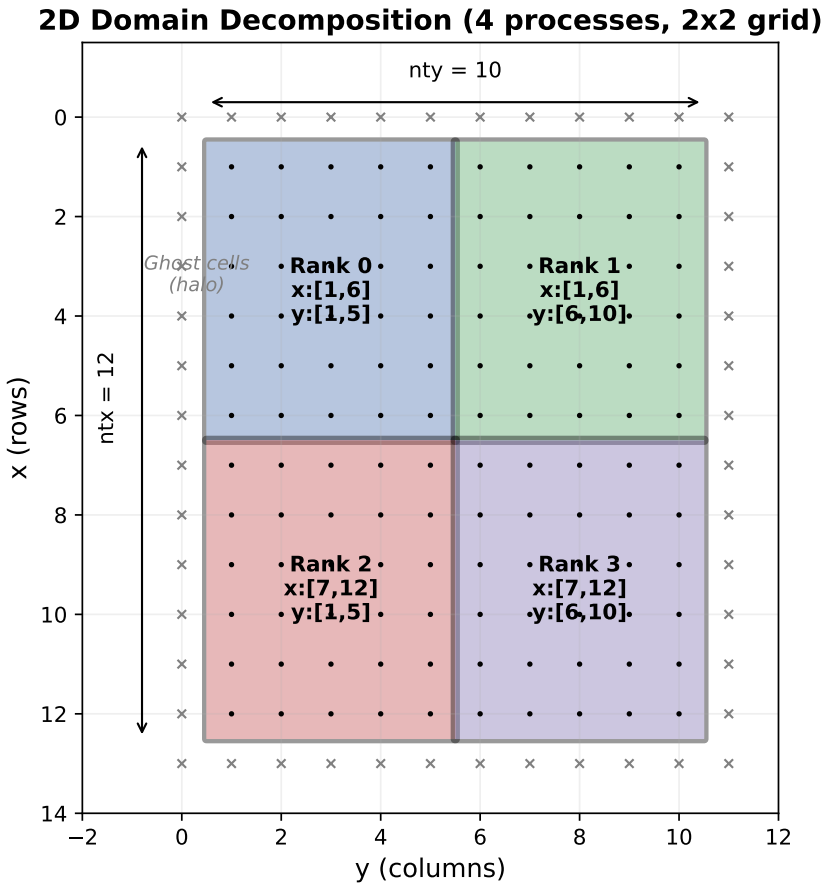

Figure 3: 2D domain decomposition for 4 processes. Each colored block shows the local subdomain with its index ranges. Gray crosses indicate ghost (halo) cells at boundaries.

3.4 Convergence

The solver converged after 278 iterations.

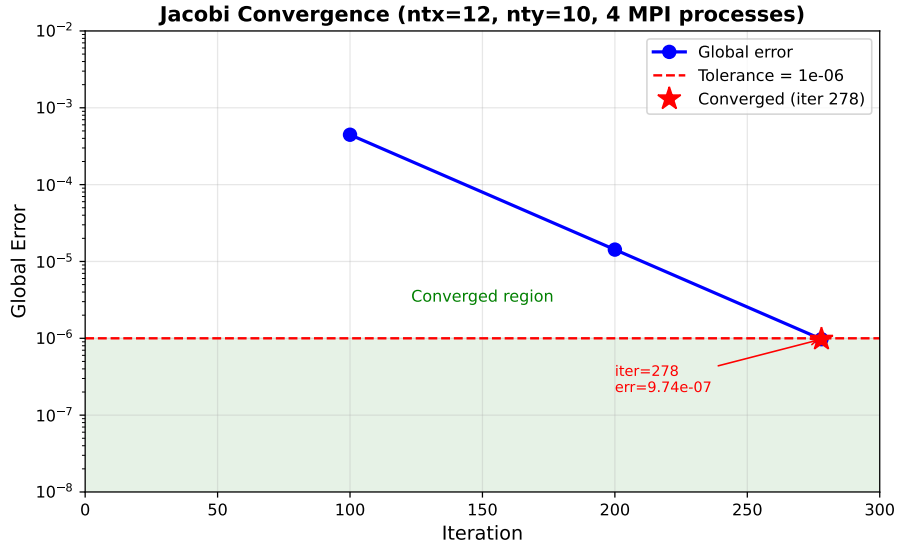

Figure 4: Jacobi convergence: global error decreases exponentially until reaching the tolerance 10^{-6}.

Iteration	Global Error
100	4.46×10^{-4}
200	1.43×10^{-5}
278	9.74×10^{-7} (converged)

Table 2: Convergence history (ntx=12, nty=10, 4 processes).

3.5 Solution Accuracy

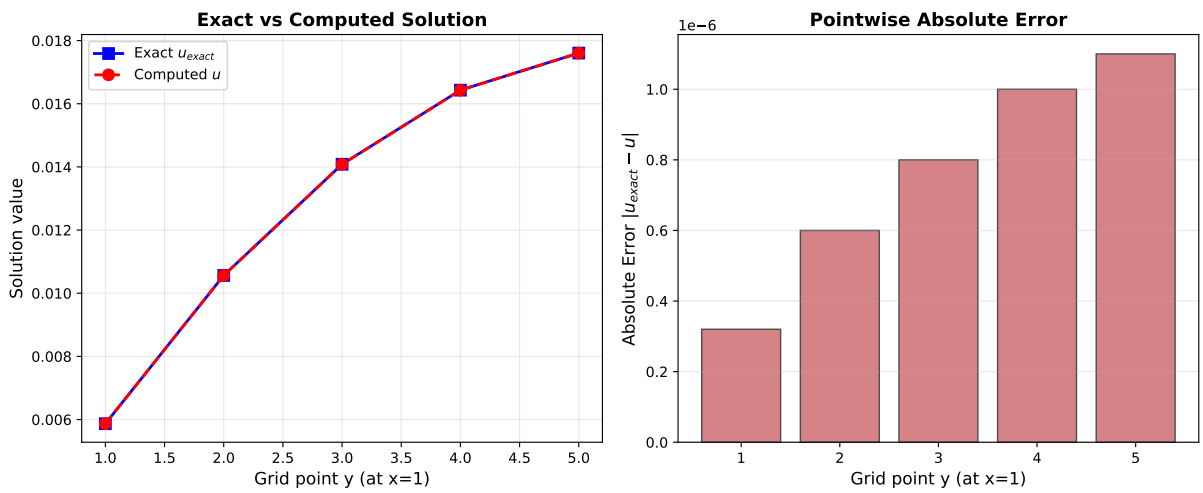

Figure 5: Left: exact vs computed solution at $x = 1$. Right: pointwise absolute error (order 10^{-6}).

The computed solution matches the exact solution to within $\sim 10^{-6}$, consistent with the convergence tolerance applied.

3.6 Observations

- The 2D decomposition reduces the surface-to-volume ratio of ghost exchanges compared to 1D strip decomposition, improving communication efficiency for large process counts.
- `MPI_PROC_NULL` (printed as -2 in OpenMPI) simplifies boundary handling: `MPI_Sendrecv` with `MPI_PROC_NULL` is a no-op, so no special-case code is needed.
- The Jacobi method converges slowly (278 iterations for a 12×10 grid). Larger grids require more iterations or a faster solver (e.g., SOR, multigrid).
- Columns are non-contiguous in row-major memory layout, therefore `MPI_Type_vector` is used for E/W ghost exchange, letting MPI handle the strided access directly without manual pack/unpack.

4 Conclusion

1. `MPI_Cart_create` provides a clean abstraction for multidimensional process topologies. Combined with `MPI_Cart_shift`, neighbor identification is automatic and handles both periodic and non-periodic boundaries.
2. Derived datatypes (`MPI_Type_vector`) are essential for exchanging non-contiguous data (columns) without manual packing.
3. 2D decomposition is preferable to 1D for large process counts: it reduces the ghost layer size from $O(N)$ to $O(\sqrt{N})$ per process.
4. The Game of Life (periodic) and Poisson (non-periodic) exercises illustrate both boundary condition modes of the Cartesian communicator.